Ain Shams University — Faculty of Engineering
Computer Engineering and Software Systems

From Pixels to Language

Milestone 1: Data Understanding, Classification and Encoder Training

CSE485 / CSE488 — Deep Learning

Team **XX**

Name	Student ID
Abdelrahman Mohamed Zayed	23P0094
Member 2	—
Member 3	—
Member 4	—
Member 5	—

Instructors: Dr. Mahmoud Khalil, Eng. Seif Yasser

Code: `https://github.com/Abdelrahmanzay/cub-pixels-to-language`
Experiment tracking (W&B):
`https://forge.coreweave.com/wandb/23p0094-ain-shams-university/cub-pixels-to-language`

November 2026

Contents

1 Introduction **2**

2 Data: EDA, Cleaning, Preprocessing and Augmentation **2**

2.1 Splits . . . 2
2.2 Class distribution . . . 3
2.3 Image sizes and quality . . . 4
2.4 Duplicates and leakage . . . 5
2.5 Captions: analysis and cleaning . . . 6
2.6 Augmentation strategy . . . 7
2.7 How the EDA informed modelling . . . 9

3 Models **9**

4 Experiments **9**

4.1 From scratch vs. transfer learning . . . 10
4.2 Architecture comparison . . . 11

5 Hyper-parameter tuning **12**

6 Experiment tracking **14**

7 Evaluation **14**

8 Explainability (Grad-CAM) **15**

9 Benchmarking **18**

10 Final selection **19**

11 Failures and limitations **19**

12 Reproducibility and interface **19**

13 Team contributions **20**

1 Introduction

The project builds one multimodal system on a single visual encoder: the encoder is first trained as a fine-grained image classifier (Milestone 1, this report), and will later be reused as the backbone of an image-colorization network and of an image-captioning model (Milestone 2).

Dataset. We use the default dataset, Caltech-UCSD Birds-200-2011 (CUB-200-2011) [1] with the natural-language descriptions of Reed et al. [2], loaded from Hugging Face (`Multimodal-Fatima/CUB_train` / `CUB_test`). It contains 11,788 photographs of 200 bird species (5,994 train / 5,794 test), each with ten human-written captions. We kept it because it satisfies all three tasks with one dataset: a discrete label per image (classification), good-quality colour photographs from which grayscale/colour pairs can be generated (colorization), and ten captions per image (captioning). It is also small enough (≈1.1 GB) for free Kaggle/Colab T4 GPUs.

Why it is hard. CUB is a *fine-grained* problem: classes are species of the same families (25 warblers, 21 sparrows, 8 gulls, 7 terns, . . . ; Fig. 3) that differ in small details such as a wing bar or the colour of the bill, while only ∼30 training images exist per class.

Summary of results. Our from-scratch residual CNN reaches **52.5%** test accuracy; transfer learning raises this to **82.4%** (ResNet-50) and **84.6%** (ConvNeXt-Tiny, our selected model). Random search found a better short-run configuration than grid search with the same budget, but the gain did not survive a full-length training run (Section 5).

2 Data: EDA, Cleaning, Preprocessing and Augmentation

All numbers below are produced by `python -m src.cli eda` and stored in `outputs/eda/eda_summary.json`; figures are in `outputs/eda/figures/`.

2.1 Splits

CUB provides no validation split, and tuning on the test set would make the reported test scores optimistic. We therefore carve a **stratified 10% validation split** (3 images per class, seed 42) out of the official training set. The official test set is left untouched so that our numbers remain comparable with published results (Section 9).

Split	Images	Per class	Used for
train	5,392	26–27	gradient updates
val	598	2–3	early stopping, model selection, hyper-parameter tuning
test (official)	5,794	11–30	final evaluation only

Table 1: Splits after duplicate removal (Section 2.4).

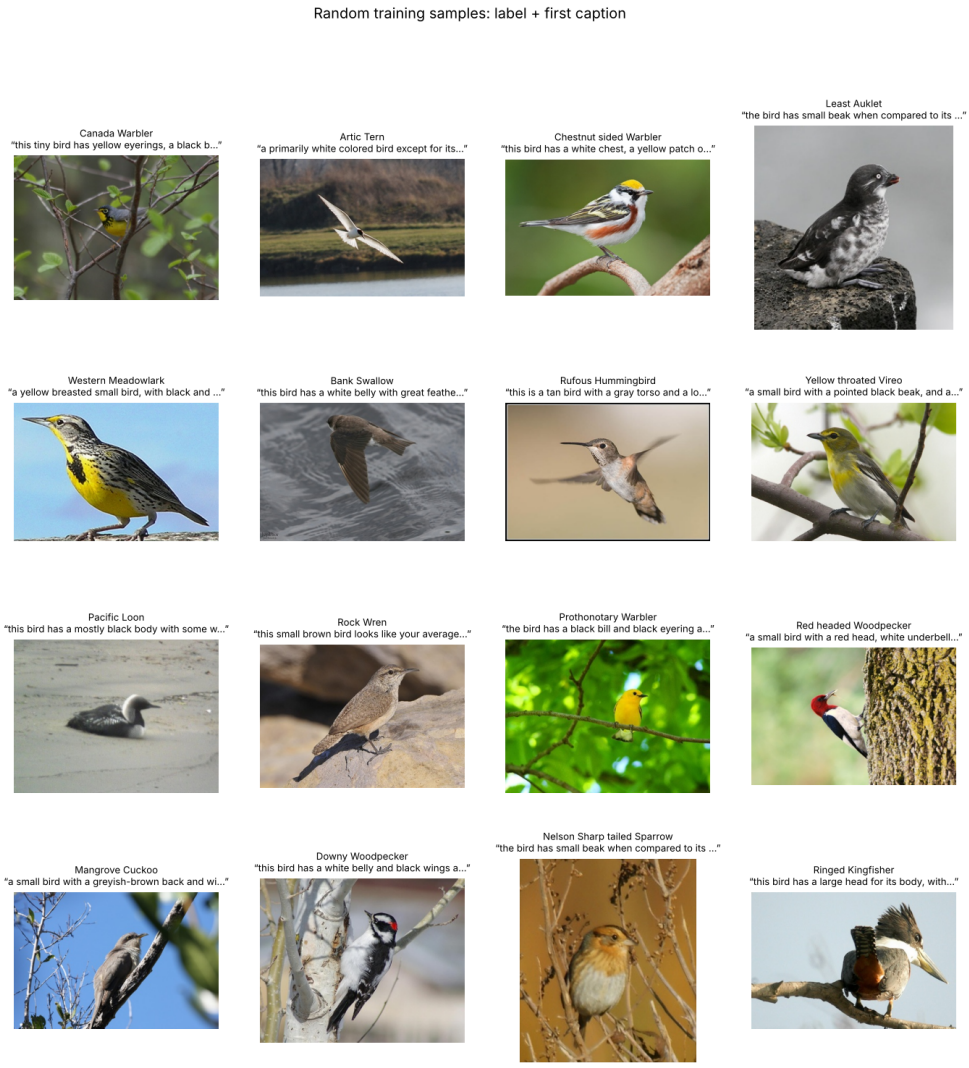


Figure 1: Random training images with their label and the first of their ten captions.

2.2 Class distribution

The training split is almost perfectly balanced (26–27 images per class), but the **test split is not**: most classes have 30 images, while a few have far fewer (Least Auklet 11, Crested Auklet 14, Spotted Catbird 15; max/min ratio 2.7). Plain accuracy is therefore dominated by the large classes, which is why we also report **macro**-averaged precision, recall and F1 (Section 7).

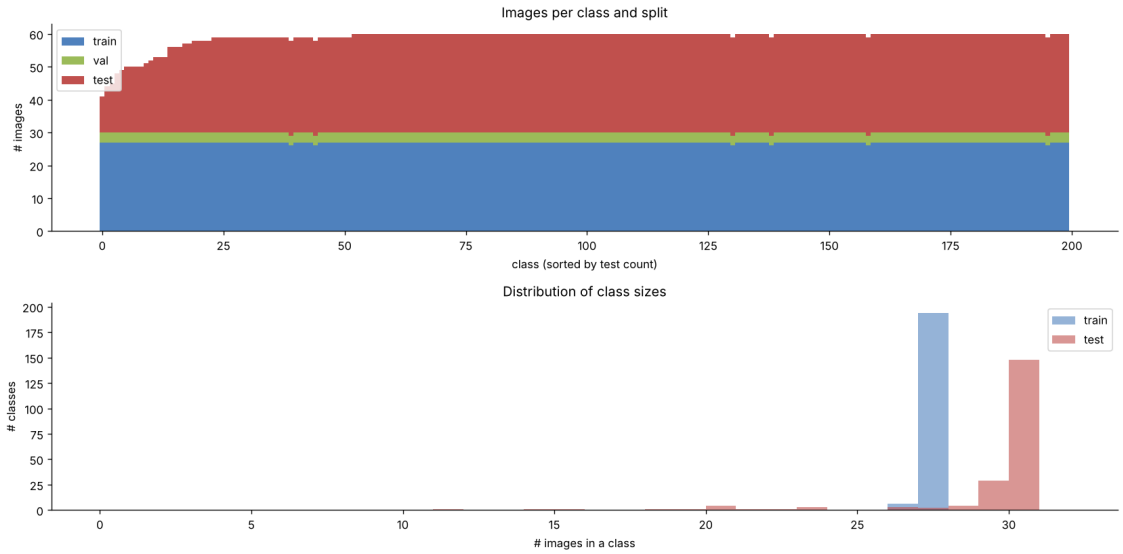


Figure 2: Top: images per class and split (sorted by test count). Bottom: distribution of class sizes.

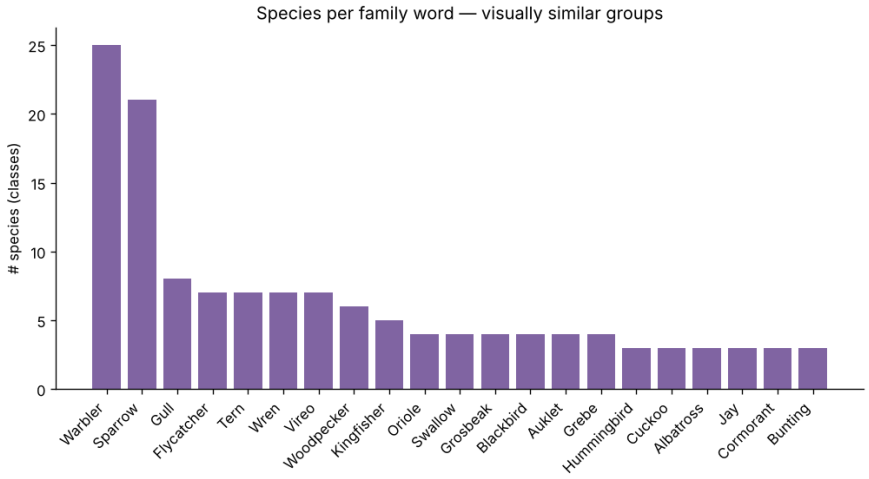


Figure 3: Number of species per family word. Large groups of visually similar species are the core difficulty of fine-grained classification.

2.3 Image sizes and quality

Images have 1,152 distinct sizes; the median is 500×375 (aspect ratio 1.33, range 0.38–3.60). 148 images have a shorter side below 224 px and are slightly upsampled at training time. Brightness, contrast, colourfulness and sharpness distributions of train and test overlap closely (Fig. 4), so there is no visible domain shift between the splits. **Preprocessing decision:** every image is resized *once* (shorter side 256 px) and cached as JPEG. Decoding full-size images every epoch was the main bottleneck on small GPUs; the cache made each epoch several times faster. Models then use 224×224 crops.

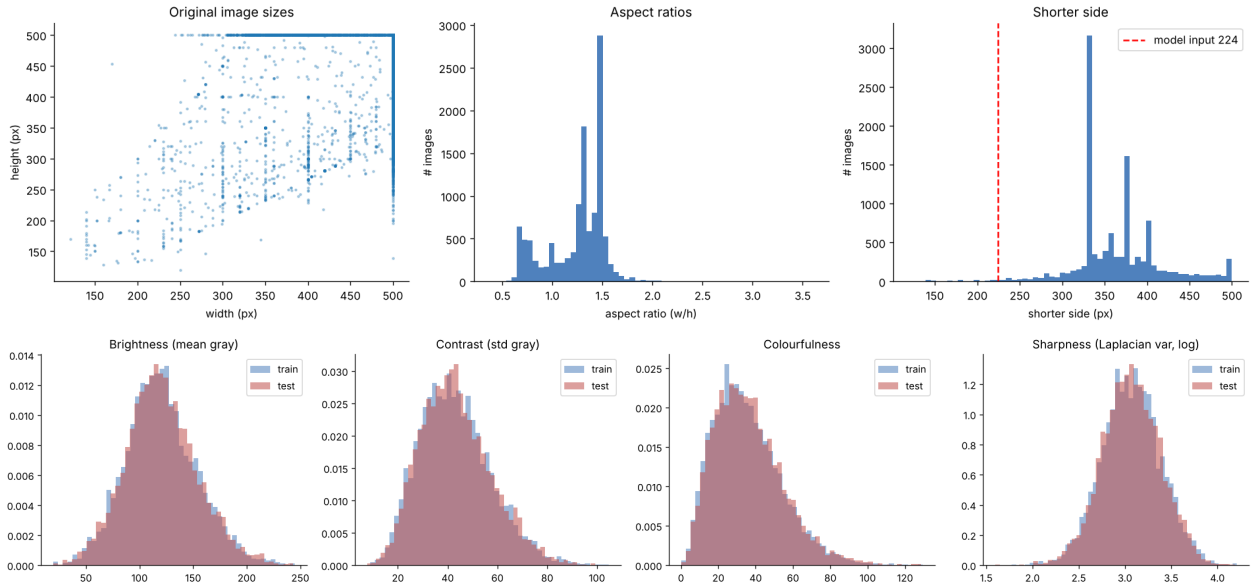


Figure 4: Image sizes (top) and per-image statistics of train vs. test (bottom).

Grayscale images. 8 images (4 train, 4 test) are stored in grayscale mode. They are kept for classification (converted to 3 channels), but they have no colour target and are flagged (`is_grayscale`) so that they are excluded from colorization in Milestone 2.

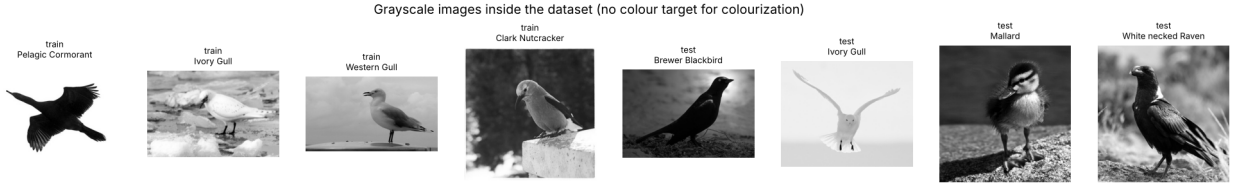


Figure 5: The eight grayscale images in the dataset.

2.4 Duplicates and leakage

Exact pixel hashing found one duplicated training image. To find re-encoded or resized copies we computed a 64-bit perceptual hash (pHash) of every image and compared all ≈69 million pairs. Of the 12 pairs within Hamming distance 4, **8 pairs at distance ≤ 2 are the same photograph** (red in Fig. 6); the pairs at distance 3–4 are different photographs with a similar composition. The 8 duplicates include:

- **2 validation↔test leaks** (Bronzed Cowbird, Prothonotary Warbler): the same image in both splits would inflate validation accuracy;
- 2 duplicates inside the training set (Pigeon Guillemot, Swainson Warbler);
- **1 label conflict**: the same photograph is labelled *Chuck-will's-widow* and *Whip-poor-will* (both in test) — a sign of annotation noise between two very similar nightjars;
- 3 same-class duplicates inside the test set.

Cleaning decision: the 4 train/val copies are dropped automatically by `prepare`; the official test set is never modified (so published numbers stay comparable), and its duplicates are only reported.

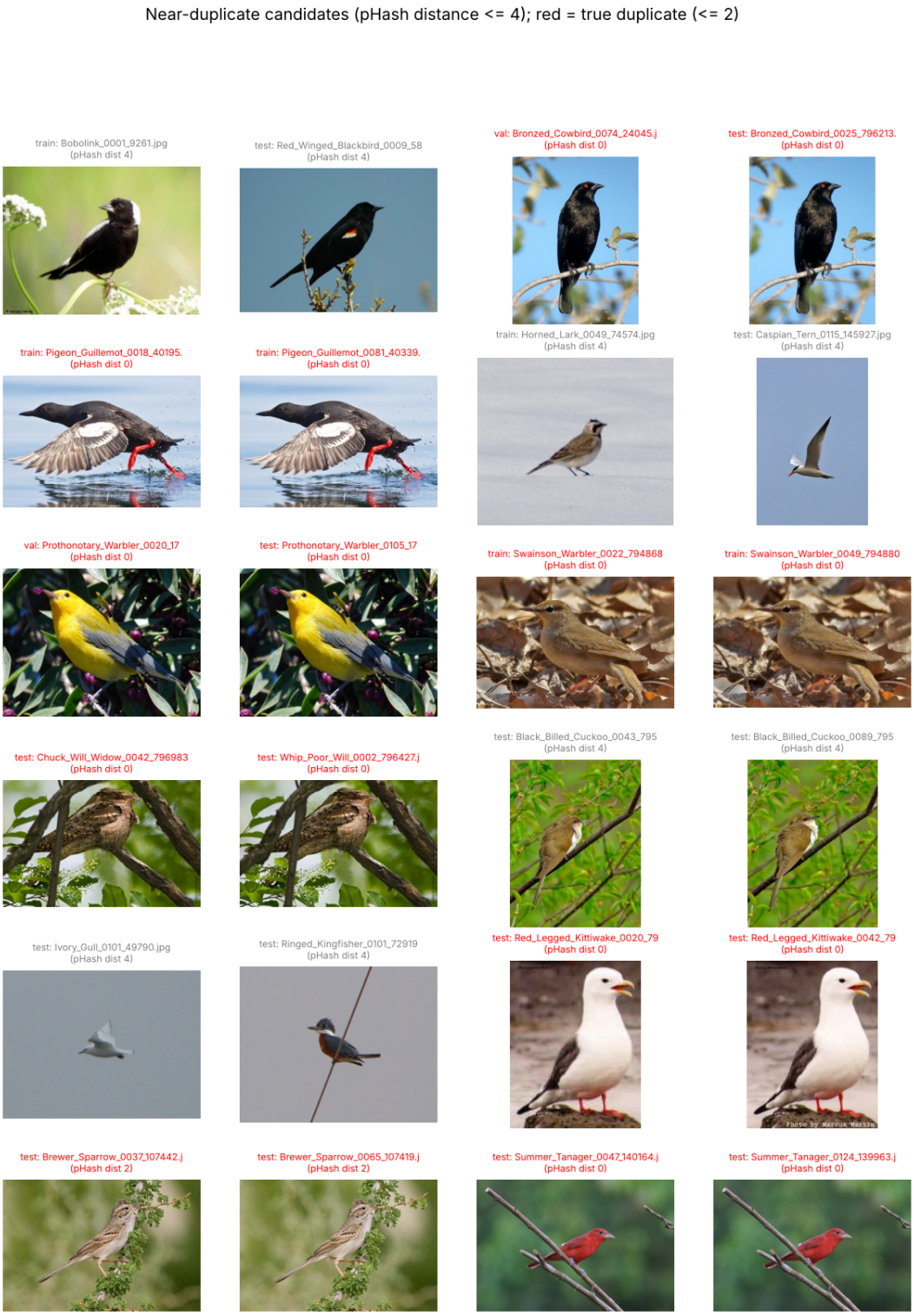


Figure 6: Duplicate candidates by perceptual hash. Red titles: true duplicates (distance ≤ 2).

2.5 Captions: analysis and cleaning

The `description` field is one string per image that holds the ten captions separated by `\n` (always 11 lines: ten captions and a trailing empty line). Our analysis found:

- **Encoding corruption:** 1,700 Unicode replacement characters (U+FFFD), almost always in pairs, standing in for spaces ("this??bird??has…") — most likely non-breaking spaces that were decoded with the wrong encoding.

- **Duplicates:** 1,306 captions repeat another caption of the same image (in 1,183 images).
- **Templated captions:** some sentences are reused across hundreds of images, e.g. "this bird has wings that are brown and has a white belly" appears 971 times. This matters for captioning: a decoder can reach a decent BLEU score by producing such generic sentences.
- **Typos and merged words:** 679 rare words are not in an English dictionary (e.g. "abdoment", "ahwite", "baclk", "ablack", "andthe"); they are only 0.09% of all tokens.
- **Uninformative captions:** 84 captions mention neither a colour nor a body part ("a bird that lives by the ocean. it eats anything."); conversely 98.7% of captions contain a colour word, confirming that colour is central to how humans describe birds.
- **Formatting noise:** spaces before punctuation ("neck , except"), missing spaces after commas ("breast,vent"), slashes and stray symbols.

Cleaning pipeline (`src/data/text.py`), applied to every line: (1) Unicode NFKC normalisation and lower-casing; (2) replace every character outside `[a-z0-9 ,.'-]` — including U+FFFD — by a space; (3) fix spacing around commas and periods; (4) collapse whitespace; (5) drop empty lines and exact duplicates within an image, keeping order. The result is **116,574 captions**: 10 per image for 10,605 images, 9 for 1,064, 8 for 115 and 7 for 4. Typos are deliberately *not* auto-corrected: automatic spell-checkers mis-correct domain words (e.g. "tarsus", "rectrices"); instead the vocabulary frequency cut-off (below) maps rare misspellings to `<unk>`.

Statistics. Captions have 15.3 words on average (median 14, 95th percentile 24, 99th percentile 32, max 66). The train+val vocabulary has 4,078 words, of which 1,872 appear only once. A minimum frequency of 5 keeps 1,243 words yet covers 99.5% of all tokens (Fig. 7); we will use this cut-off and a maximum length of ≈32 tokens for the Milestone-2 caption decoder.

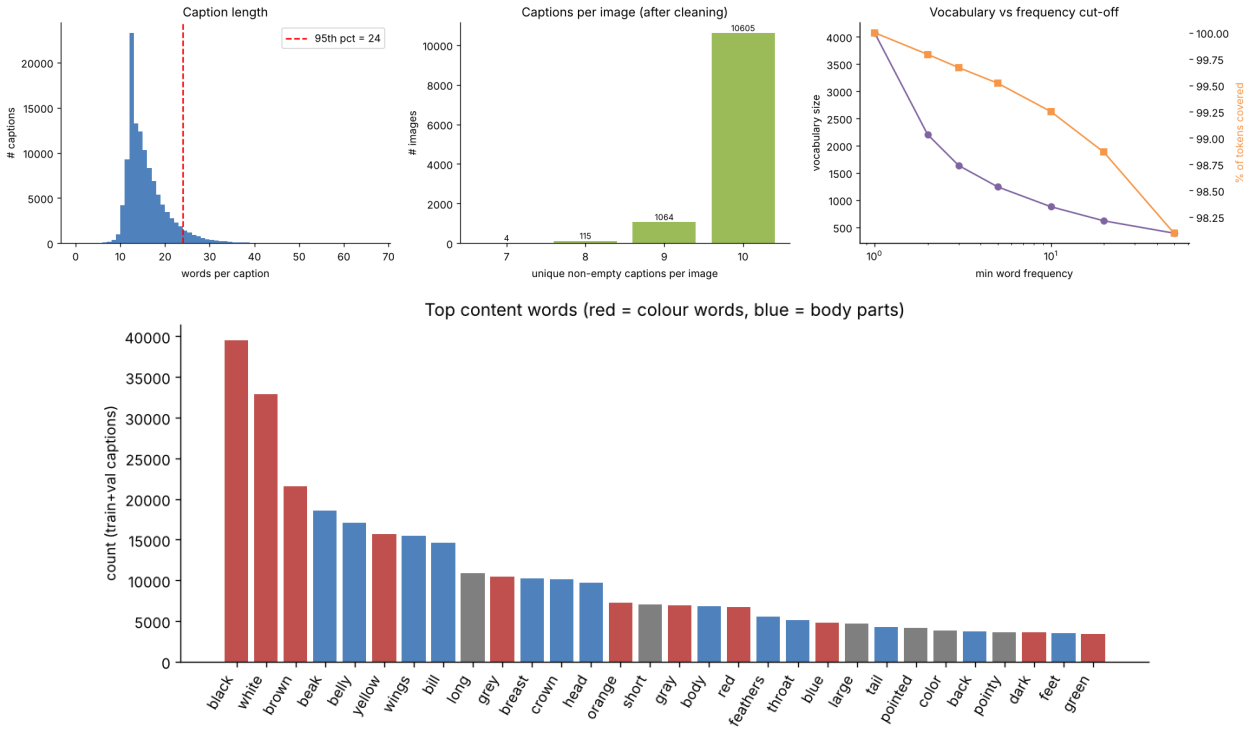


Figure 7: Caption length, captions per image after cleaning, vocabulary size vs. frequency cut-off, and the most frequent content words (red: colour words, blue: body parts).

2.6 Augmentation strategy

Augmentation targets the real nuisance factors of CUB: the bird's scale and position in the frame (RandomResizedCrop), left/right pose (horizontal flip), small camera tilt (rotation ±15°), lighting (brightness/contrast ±20%, saturation ±10%) and partial occlusion by branches (RandomErasing,

$p = 0.25$). **We never use hue jitter or random grayscale**: colour is one of the most discriminative cues between species (e.g. Scarlet vs. Summer Tanager), so changing it would create training images whose label is wrong. Three levels were implemented (Fig. 8); the *strong* level sometimes crops the bird out of the image entirely (row 3, first sample), so *standard* is the default and *light* is used for the linear probe, which has few trainable parameters.



Figure 8: Light / standard / strong augmentation on three training images.

2.7 How the EDA informed modelling

- Imbalanced test set ⇒ report macro-F1 and balanced accuracy, not only accuracy.
- Only ∼27 images per class ⇒ strong regularisation (label smoothing, weight decay, dropout, augmentation) and an expectation that transfer learning will dominate training from scratch.
- Colour is discriminative ⇒ no hue/grayscale augmentation.
- Small, fine details ⇒ higher resolution would help; we use 224 px for GPU budget reasons and discuss this in Section 11.
- Duplicates and label conflict ⇒ leak-free validation, and an explanation for one of the most confused class pairs (Section 7).

3 Models

All encoders implement one interface (`BaseEncoder` in `src/models/encoders.py`): `forward_features(x)` returns four feature maps at strides 4, 8, 16 and 32 (56, 28, 14 and 7×7 for a 224 px input), together with their channel counts and the layer used by Grad-CAM. Encoders are created through a registry, so any encoder can be selected from the config without touching the training code. The multi-scale output is designed for Milestone 2: the colorization U-Net uses all four maps as skip connections, and the caption decoder attends over the last 7×7 grid. The classifier is *encoder → global average pooling → dropout → linear layer (200 classes).*

Encoder	Params	Initialisation	Notes
Scratch CNN (ours)	11.3M	random (Kaiming)	residual, 4 stages $64-128-256-512$, 2 blocks each
Scratch CNN, no skips	11.1M	random	ablation: same CNN without residual connections
ResNet-50 [3]	23.9M	ImageNet	bottleneck residual network
EfficientNet-B0 [4]	4.3M	ImageNet	depthwise convs + squeeze-excitation, compound scaling
ConvNeXt-Tiny [5]	28.0M	ImageNet	modernised ResNet: 7×7 depthwise convs, LayerNorm, GELU

Table 2: Encoders (parameter counts include the 200-class head).

Our from-scratch CNN. Stem: 3×3 conv (stride 2) → 3×3 conv → max-pool; then four stages of two residual blocks (3×3 conv–BN–ReLU–3×3 conv–BN, plus an identity or 1×1 projection shortcut). Every stage after the first halves the resolution and doubles the channels. Weights use Kaiming initialisation, and the last BatchNorm of every residual branch is initialised to zero so that each block starts as an identity mapping, which stabilises early training.

Transfer-learning strategies. (a) *Linear probe*: the ImageNet backbone is frozen (including its BatchNorm statistics) and only the linear head is trained. (b) *Full fine-tuning*: all layers are trained, with *discriminative learning rates* — the head uses the base LR and the pretrained backbone uses $0.1\times$ that LR, so the useful ImageNet features are adapted gently instead of being overwritten.

4 Experiments

Common setup. AdamW, linear warm-up then cosine decay (per step), label smoothing 0.1, mixed precision (FP16), batch size 32 (64 for the scratch CNNs), 224×224 inputs, ImageNet normalisation for pretrained models and dataset statistics (mean $[0.485, 0.499, 0.432]$) for the scratch models, early stopping on validation accuracy with the best checkpoint restored. Hardware: one NVIDIA Tesla T4 (Kaggle). One run per configuration, seed 42.

Run	LR (head)	Weight decay	Aug.	Epochs (best)	Time	s/epoch
Scratch CNN	2e-3	0.05	standard	80 (64)	23.8 min	17.8
Scratch CNN, no skips	2e-3	0.05	standard	40 (37)	11.8 min	17.7
ResNet-50 linear probe	3e-3	1e-4	light	15 (11)	5.1 min	20.2
ResNet-50 fine-tune	1e-3	1e-4	standard	30 (27)	9.9 min	19.7
ResNet-50 fine-tune, tuned	2.09e-3	3.4e-5	standard	30 (23)	11.7 min	23.4
EfficientNet-B0 fine-tune	1e-3	1e-4	standard	30 (29)	10.4 min	20.8
ConvNeXt-Tiny fine-tune	1e-3	1e-4	standard	28 (20)	13.5 min	28.8

Table 3: Settings of every training run. Pretrained backbones use 0.1× the head LR.

Model	Accuracy	Top-5	Precision	Recall	F1	Best val acc
Scratch CNN, no skips	36.5	67.0	35.5	36.7	35.0	36.0
Scratch CNN (ours)	52.5	76.7	53.2	52.5	52.1	51.7
ResNet-50 linear probe	67.6	91.2	69.0	68.0	67.7	66.4
EfficientNet-B0 fine-tune	79.8	94.4	80.3	80.0	79.8	79.1
ResNet-50 fine-tune	82.4	95.2	82.8	82.5	82.3	81.6
ResNet-50 fine-tune, tuned	82.4	95.3	82.8	82.6	82.3	81.8
ConvNeXt-Tiny fine-tune	**84.6**	**96.0**	**85.0**	**84.8**	**84.5**	**84.3**

Table 4: Test-set results (%, official CUB test split, 5,794 images). Precision/recall/F1 are macro-averaged.

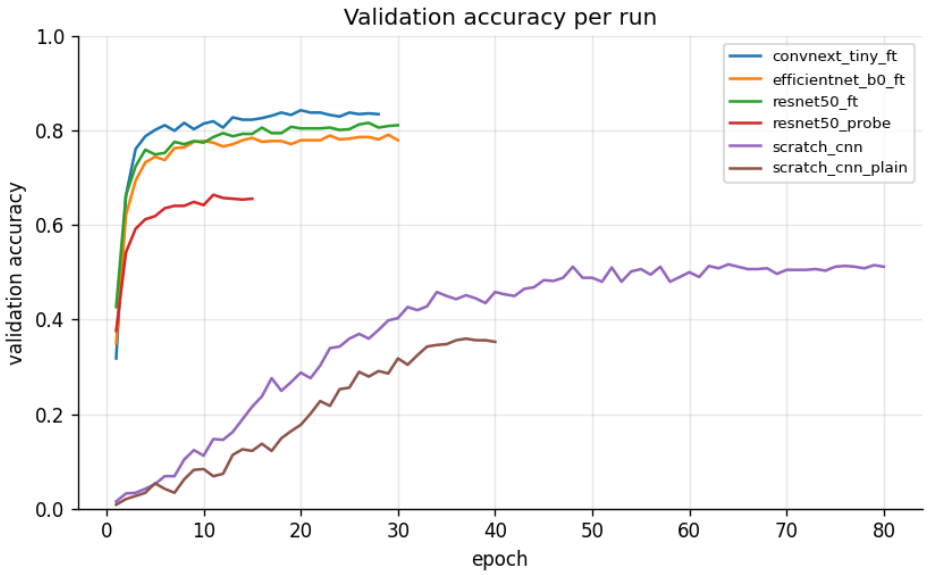


Figure 9: Validation accuracy per epoch for all runs.

4.1 From scratch vs. transfer learning

Transfer learning wins by a wide margin: the best fine-tuned model is **32 points** more accurate than our scratch CNN, converges in 20 epochs instead of 64, and is already at ∼75% validation accuracy after 3 epochs (Fig. 9). With only ∼27 images per class, a network trained from scratch must learn edges, textures and bird parts from about 5,400 images; ImageNet pretraining provides these generic features for free.

The scratch CNN also shows the clearest **overfitting** (Fig. 10): training accuracy reaches 96% while validation accuracy plateaus at 51%, and the validation loss stops improving after epoch ∼45 while the training loss keeps falling. Strong regularisation (weight decay 0.05, dropout 0.3, label

smoothing, RandomErasing) reduced but could not close this gap. The pretrained models also fit the training set ($> 98\%$), but their gap is much smaller because their features generalise.

Linear probe vs. fine-tuning. Frozen ImageNet features already give 67.6%, i.e. they are informative about birds; but fine-tuning the whole backbone adds **+14.8 points**. ImageNet features are tuned to separate e.g. "bird vs. dog", not "California Gull vs. Western Gull", so adapting the deeper layers to fine-grained cues is essential.

Ablation: residual connections. With the same 11M-parameter CNN, removing the skip connections lowers validation accuracy at epoch 40 from 45.8% to 35.3%. Crucially the no-skip network does *not* overfit — its training accuracy is only 44% — it is harder to *optimise*. This is the degradation problem described by He et al. [3]: residual connections give gradients a short path through the 18-layer network.

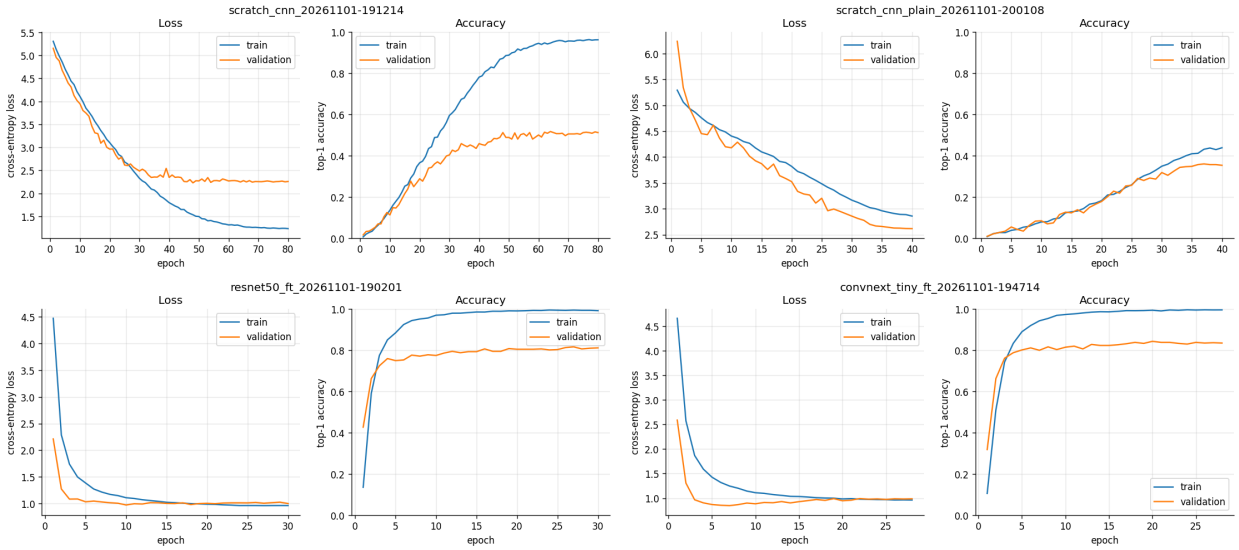


Figure 10: Training vs. validation loss and accuracy. Top: scratch CNN with and without residual connections. Bottom: fine-tuned ResNet-50 and ConvNeXt-Tiny.

4.2 Architecture comparison

Among pretrained backbones, ConvNeXt-Tiny is best (84.6%), ResNet-50 second (82.4%) and EfficientNet-B0 third (79.8%), which matches their ordering on ImageNet and the finding of Kornblith et al. [8] that better ImageNet architectures transfer better. EfficientNet-B0 is the most parameter-efficient (79.8% with 4.3M parameters, 6.6× fewer than ConvNeXt), while ConvNeXt costs about 45% more time per epoch than ResNet-50.

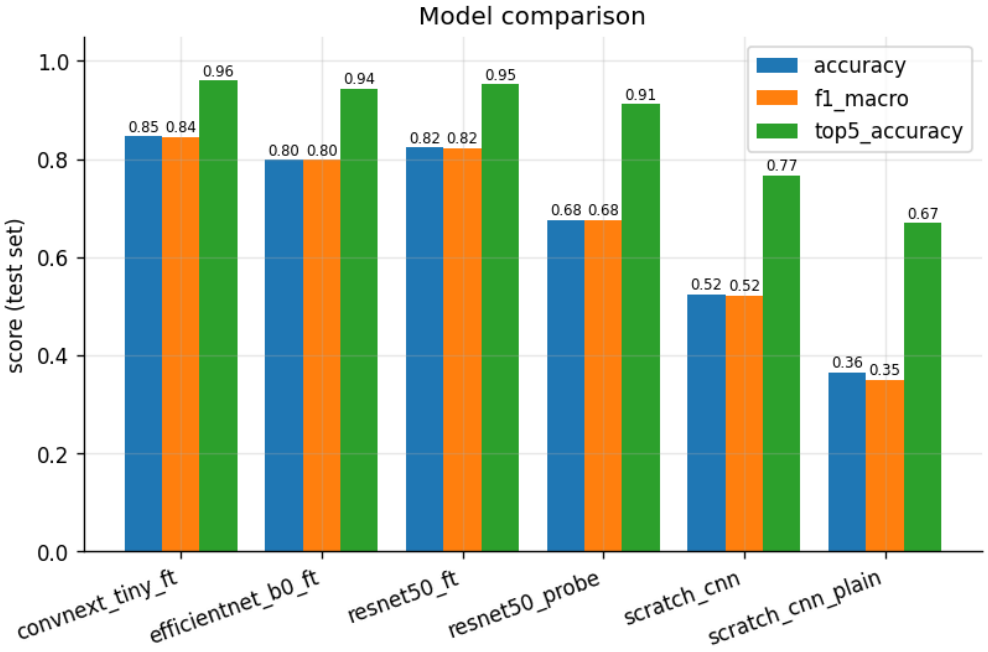


Figure 11: Test accuracy, macro-F1 and top-5 accuracy of all models.

5 Hyper-parameter tuning

Setup. We tuned the two hyper-parameters with the largest expected effect for fine-tuning, the **learning rate** (head LR; backbone = 0.1×) and **weight decay**, on ResNet-50 full fine-tuning. Each trial is a short 8-epoch run (enough to rank configurations); selection uses only the validation set.

- **Grid search:** LR $\in \{3\times10^{-4}, 10^{-3}, 3\times10^{-3}\}$ × WD $\in \{10^{-5}, 10^{-4}, 10^{-3}\}$ = 9 trials.
- **Random search [7]:** 9 trials (same budget), LR $\sim$ log-uniform$[2\times10^{-4}, 5\times10^{-3}]$, WD $\sim$ log-uniform$[10^{-5}, 10^{-2}]$, i.e. the same ranges.

Method	Trials	Best val acc	Total time	Best found at
Grid	9	79.3% (LR 3e-3, WD 1e-4)	25.3 min	trial 8
Random	9	**80.4%** (LR 2.09e-3, WD 3.4e-5)	24.3 min	trial 8

Table 5: Comparison of tuning methods with the same budget.

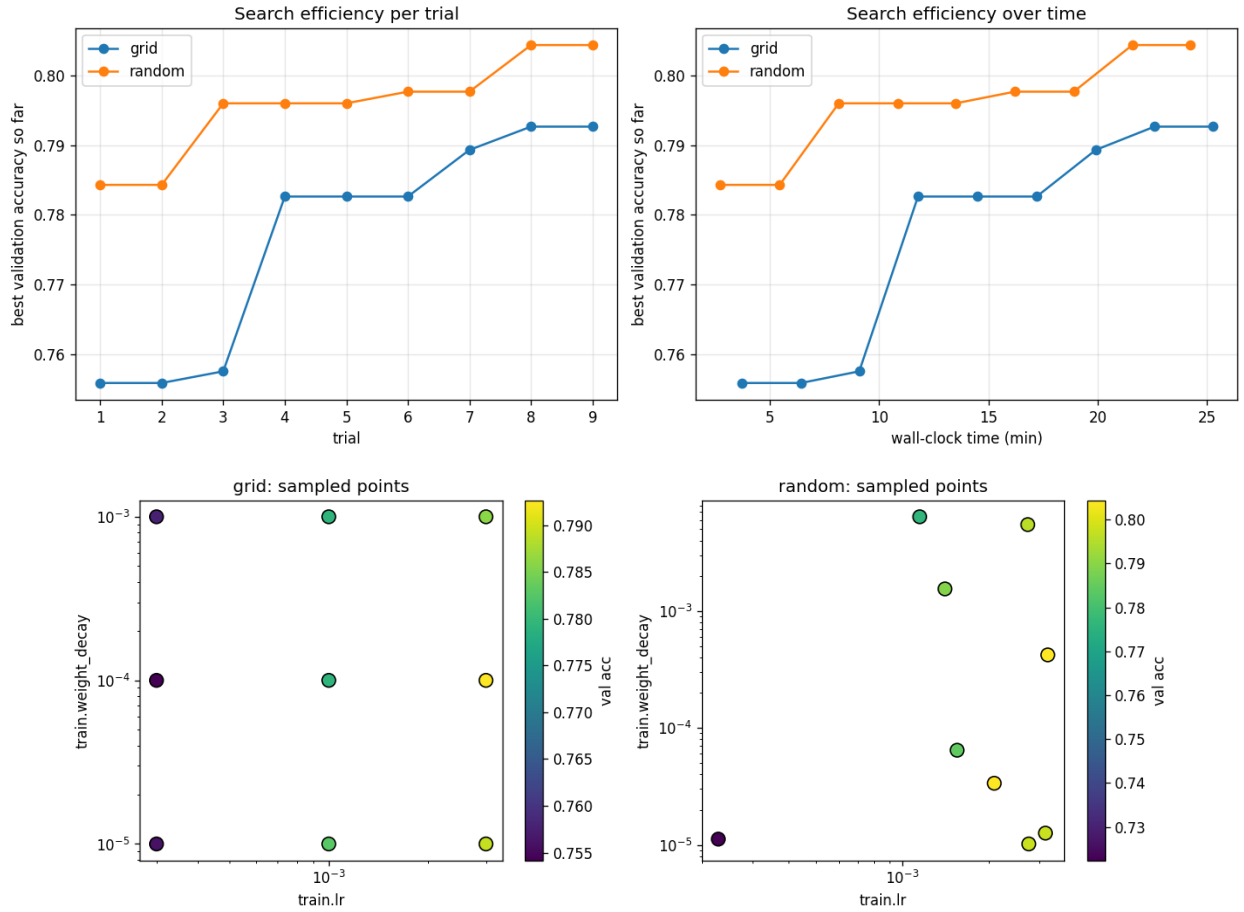


Figure 12: Top: best validation accuracy so far vs. trial number and wall-clock time. Bottom: where each method sampled (colour = validation accuracy).

Findings.

1. **Learning rate dominates; weight decay barely matters.** In the grid, the mean accuracy per LR row is 75.6%, 78.0% and 78.9% for LR 3×10^{-4}, 10^{-3} and 3×10^{-3}, while changing WD by 100× moves accuracy by less than 0.7 points.
2. **Random search was better with the same budget** (80.4% vs. 79.3%, and ahead after every trial). Grid search spends three trials on each LR value; random search tries 9 different LR values, which is more useful when only one of the two dimensions is important [7]. The grid's best LR is also on the edge of its range, a sign the grid was too narrow; random search covered more of the LR axis.
3. **Caveat: noise.** With 598 validation images one image is 0.17%, and the standard error of an accuracy near 80% is about $\sqrt{0.8\cdot0.2/598}\approx1.6\%$. Differences of ∼1 point between trials are within noise.

Baseline vs. tuned model. Retraining ResNet-50 for the full 30 epochs with the best random-search configuration gave 82.40% test accuracy vs. 82.38% for the default configuration — **no meaningful improvement**. The short 8-epoch proxy rewards hyper-parameters that converge *fast* (a higher LR), but with a full cosine schedule the default LR catches up. We report this honestly as a limitation of low-fidelity tuning: the proxy should be closer to the real schedule (e.g. 15–20 epochs) or combined with a multi-fidelity method such as Hyperband.

6 Experiment tracking

Every run is logged with Weights & Biases [14] through a small `Tracker` interface (`src/tracking/trackers.py`); a local tracker additionally writes `config.yaml`, `history.jsonl`, `summary.json` and the plots to each run folder, so all figures can be regenerated offline. For each run we log: the full configuration (all hyper-parameters, encoder, augmentation, seed), per-epoch train/validation loss and accuracy, validation macro-F1 and top-5, learning rate and epoch time, the final test metrics, parameter counts, the device, and the path of the best checkpoint. Tuning trials are grouped (`tune-grid-*`, `tune-random-*`) and tagged, so the two searches can be compared side by side. In total the project contains 25 runs (7 training runs and 18 tuning trials).

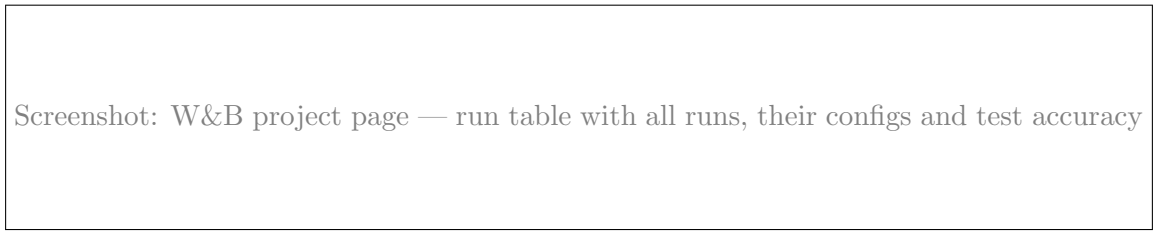


Figure 13: W&B: all runs of Milestone 1.

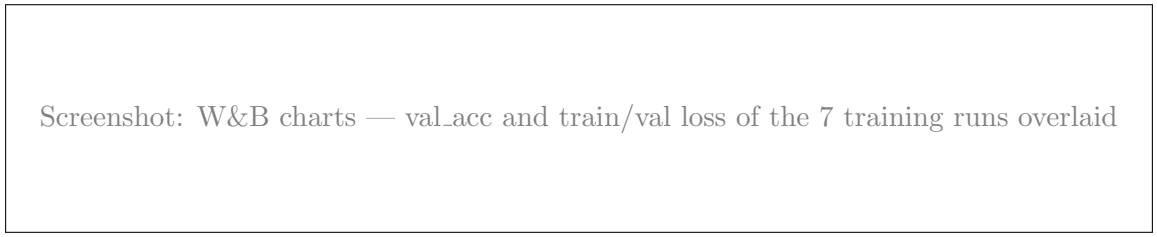


Figure 14: W&B: learning curves of the training runs.

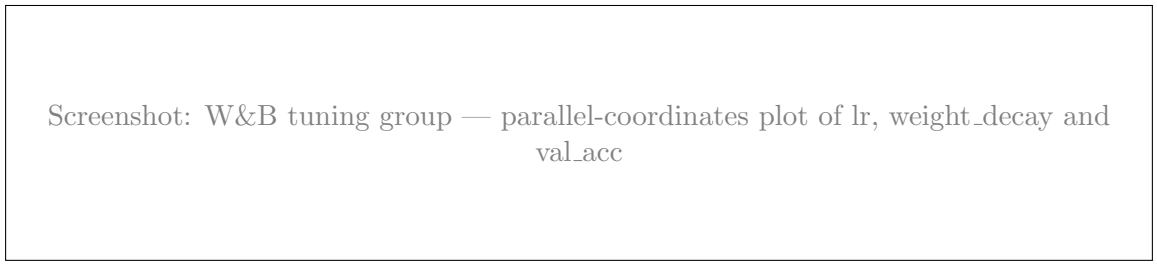


Figure 15: W&B: hyper-parameter search trials.

7 Evaluation

Metrics.

- **Accuracy**: fraction of test images whose top-1 prediction is correct. Simple, but dominated by the larger classes of the imbalanced test set.
- **Top-5 accuracy**: the true class is among the five most probable. In fine-grained tasks the top-2/3 are often sibling species, so top-5 shows whether the model at least narrows the answer to the right group.
- **Macro precision / recall / F1**: computed per class and averaged with equal weight, so the small classes (e.g. Least Auklet, 11 images) count as much as the large ones. Precision: of the images predicted as class c, how many are c; recall: of the images of class c, how many were found; F1: their harmonic mean.

- **Balanced accuracy**: mean per-class recall.
- **Confusion matrix** and the list of the most confused class pairs.

Interpretation. For every model, macro-F1 is within 0.5 points of accuracy (Table 4), so the test imbalance does not hide a failure on small classes. ConvNeXt's top-5 accuracy of 96.0% means that for 96% of images the correct species is among five candidates — the remaining errors are mostly between siblings. Its per-class accuracy ranges from 100% (e.g. the woodpeckers with distinctive patterns) down to 23% (California Gull, Common Tern).

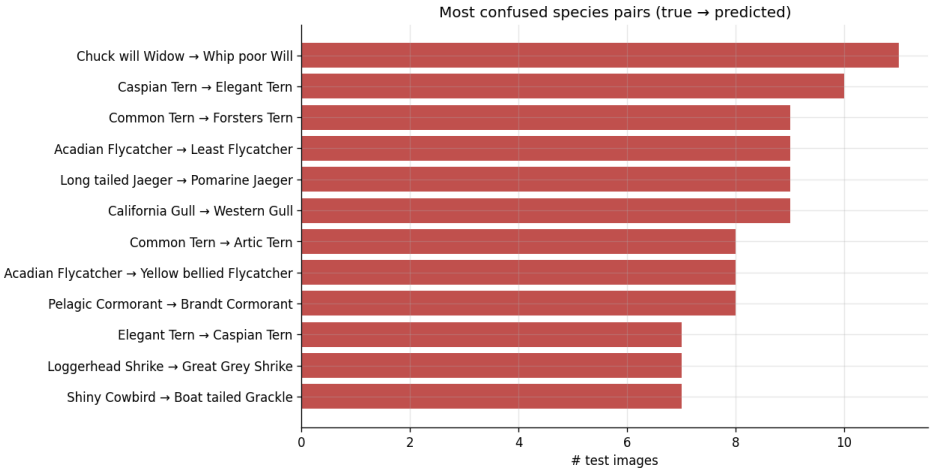


Figure 16: Most confused (true → predicted) pairs of ConvNeXt-Tiny on the test set.

The most confused pairs (Fig. 16) are exactly the hard fine-grained cases: terns (Caspian/Elegant, Common/Forster's/Arctic), gulls (California/Western), *Empidonax* flycatchers (Acadian/Least/Yellow-bellied) and jaegers — species that birders separate by size, voice or subtle bill and wing-tip details that are hard to see at 224 px. The top pair, **Chuck-will's-widow → Whip-poor-will** (11 of 26 test images), is the same pair for which the EDA found an identical photograph labelled as both species — evidence that the labels of these two nightjars are themselves noisy.

8 Explainability (Grad-CAM)

We implemented Grad-CAM [6] (`src/visualization/gradcam.py`) with forward/backward hooks on the last stage of each encoder. For the target class c, the weight of channel k is the spatial mean of $\partial y^c / \partial A^k$, and the map is $\mathrm{ReLU}(\sum_k \alpha_k^c A^k)$, upsampled to the image size. The same test images are shown for the scratch CNN, ResNet-50 and ConvNeXt; images are selected from the scratch model's predictions (6 random correct ones and its 6 most confident mistakes).

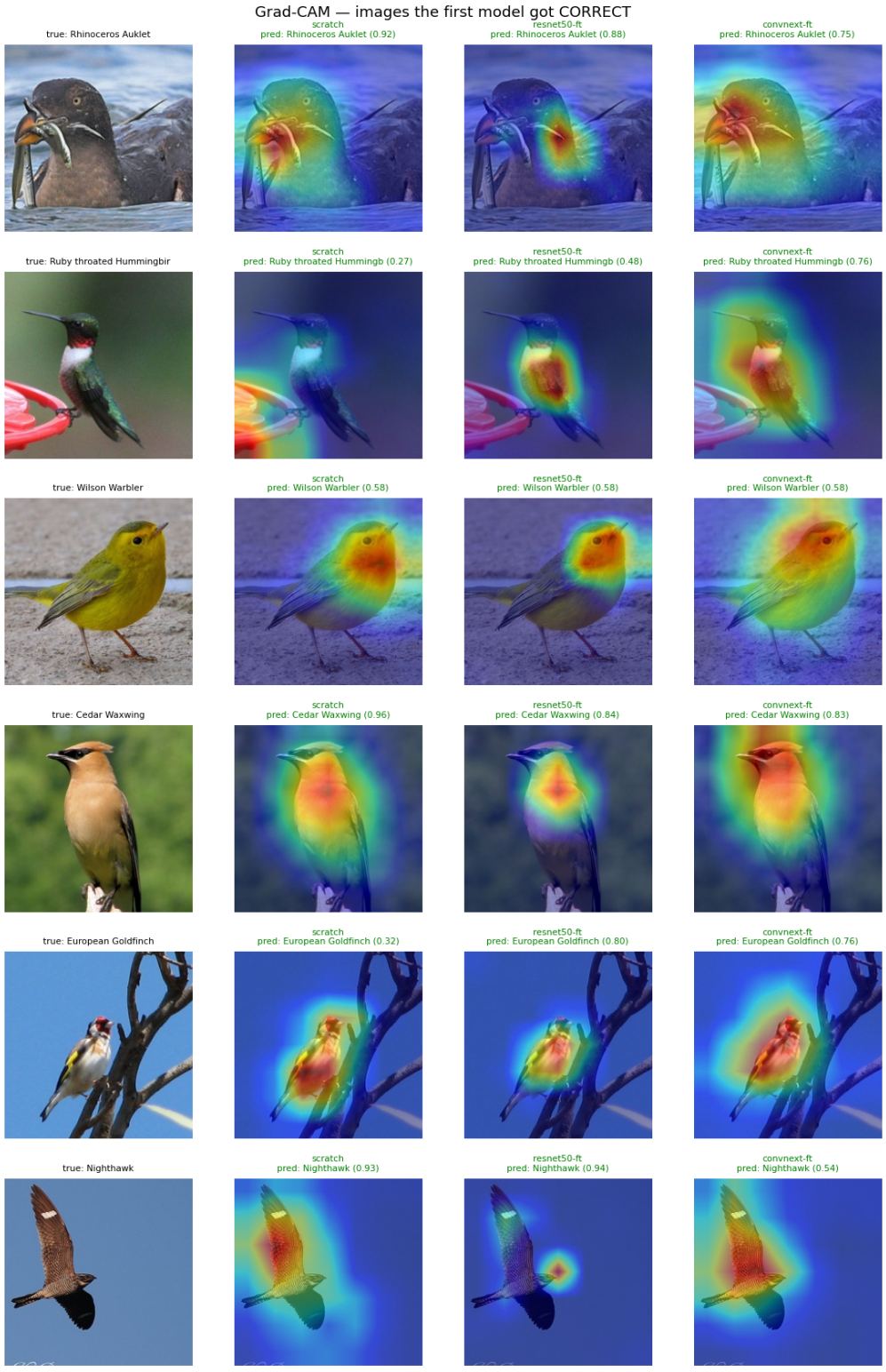


Figure 17: Grad-CAM on images the scratch CNN classified correctly (green: correct, red: wrong; confidence in brackets).

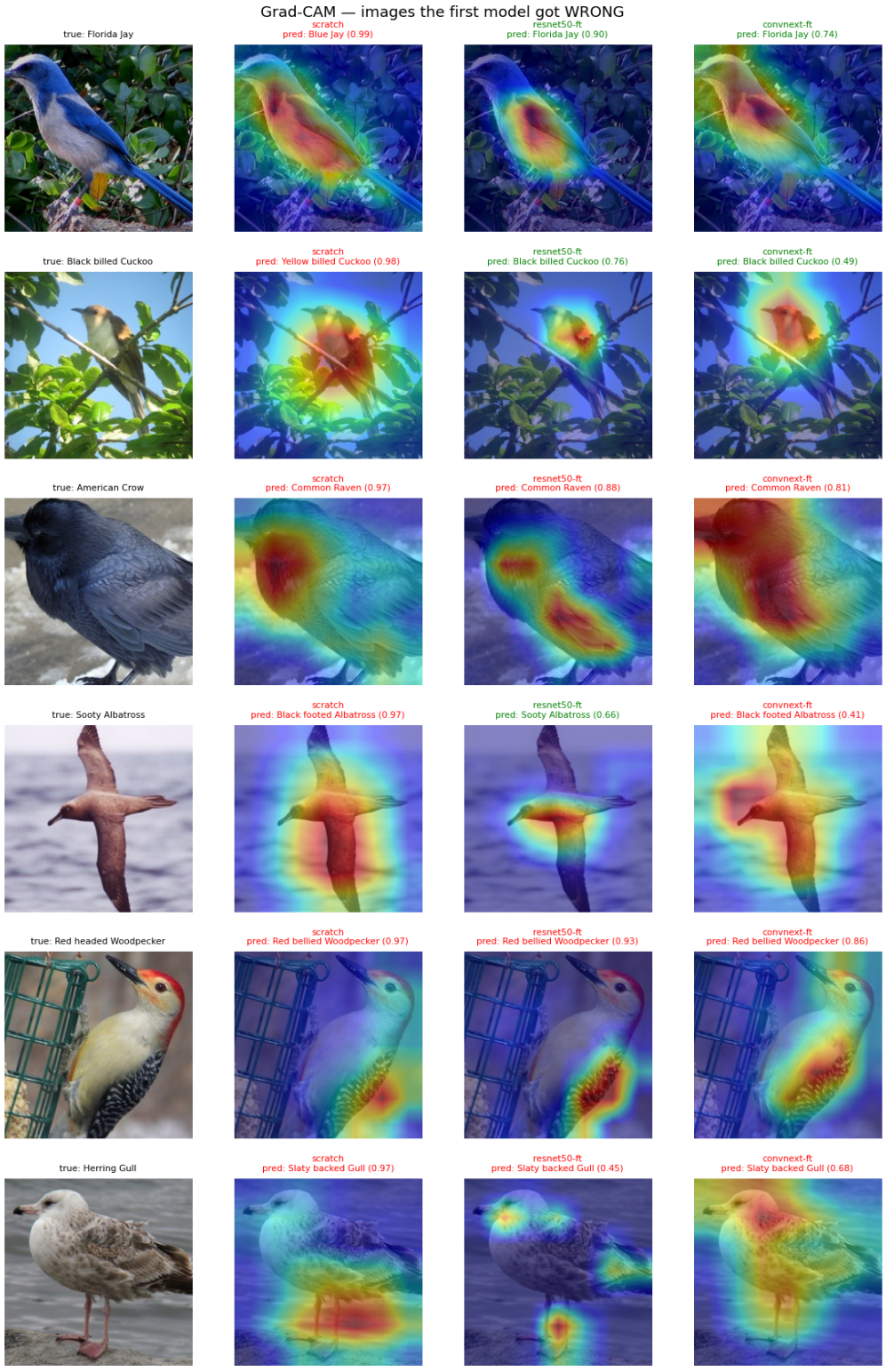


Figure 18: Grad-CAM on the scratch CNN's most confident mistakes.

Analysis.

- **Correct predictions** (Fig. 17): all models mostly look at the bird, typically the head and breast, where most field marks are (e.g. the Wilson's Warbler's head, the Cedar Waxwing's face and crest). The pretrained models produce tighter, part-focused maps (ResNet-50 on the hummingbird's red throat), while the scratch CNN's maps are broader.
- **Right for the wrong reason:** on the Ruby-throated Hummingbird the scratch CNN's attention is on the *red feeder*, not on the bird (and its confidence is only 0.27). It has learned a context

shortcut — red objects near hummingbirds — that will not generalise; the pretrained models attend to the bird itself.

- **Scratch mistakes are confident and coarse** (Fig. 18): it predicts Blue Jay for a Florida Jay with 0.99 confidence while attending to the whole body; the fine-tuned models, attending to the same region but with better features, get it right. For the Herring Gull the scratch CNN looks at the ground and legs.
- **Mistakes shared by all models are informative:** American Crow → Common Raven is wrong for all three: the two are separated mainly by size and bill thickness, which a single cropped image does not show reliably. The Red-headed Woodpecker is predicted as Red-bellied Woodpecker by all models, whose attention lies on the barred back/belly — shared by both species — rather than on the fully red head that distinguishes it.

9 Benchmarking

We compare against published results on the same benchmark: the official CUB-200-2011 test split (5,794 images, which we did not modify) and the same metric (top-1 accuracy). The open-source pretrained backbones are additionally evaluated by us on our split with our metrics (Table 4).

Method	Backbone (pretraining)	Input	Top-1 (%)	Source
Trained from scratch				
ResNet-50, 300 epochs†	ResNet-50 (none)	–	66.7	[12]
Ours, scratch CNN, 80 epochs	11M CNN (none)	224	**52.5**	
Fine-tuned ImageNet backbones				
ResNet-50 (+2 FC layers)	ResNet-50 (IN-1k)	224	77.5	[11]
ResNet-50 baseline	ResNet-50 (IN-1k)	448	84.5	[9]
ResNet-50 baseline	ResNet-50 (IN-1k)	448	85.3	[10]
NTS-Net (part-based)	ResNet-50 (IN-1k)	448	87.5	[9]
TransFG (state of the art class)	ViT-B/16 (IN-21k)	448	91.7	[13]
Ours, ResNet-50 fine-tune	ResNet-50 (IN-1k)	224	**82.4**	
Ours, ConvNeXt-Tiny fine-tune	ConvNeXt-T (IN-1k)	224	**84.6**	

Table 6: Comparison with published results on the CUB-200-2011 test split. †Reported by the authors as "validation accuracy".

Discussion of the differences.

- **Resolution** is the largest factor. Standard fine-grained baselines use 448×448 inputs; at 224 px our fine-tuned ResNet-50 (82.4%) is about 2–3 points below the 448 px ResNet-50 baselines (84.5–85.3%), consistent with small field marks being lost at lower resolution. At the same 224 px resolution our ResNet-50 is well above the 77.5% reported in [11]; part of this is our training recipe (AdamW, discriminative LRs, label smoothing, augmentation) and the stronger torchvision ImageNet weights (`IMAGENET1K_V2`) we start from.
- **ConvNeXt-Tiny at 224 px matches ResNet-50 at 448 px** (84.6% vs. 84.5%) with about four times fewer input pixels, showing the benefit of a newer architecture.
- **The gap to the state of the art** (TransFG, 91.7%) comes from three things we do not use: a ViT-B/16 pretrained on the 14× larger ImageNet-21k, 448 px input, and explicit part-selection mechanisms designed for fine-grained recognition.
- **From scratch**, our CNN is below the 300-epoch ResNet-50 of [12] (52.5% vs. 66.7%); that model is twice as large and trained almost four times longer. Our training had not fully saturated, but was strongly overfitting.
- **Limits of the comparison**: we report a single seed, while published baselines differ in epochs, optimisers and augmentation; these numbers should be read as reference points, not as a controlled comparison.

10 Final selection

Classifier: ConvNeXt-Tiny, fully fine-tuned — best on every metric (84.6% accuracy, 84.5% macro-F1, 96.0% top-5), best validation accuracy (84.3%, which is how the model was *selected*; the test set was only used for reporting), with an acceptable cost (28.8 s/epoch on a T4).

Encoder for Milestone 2. The fine-tuned ConvNeXt-Tiny encoder will be the main backbone for colorization and captioning: it provides the best bird-specific features and four feature maps (96–768 channels, strides 4–32) that plug directly into a U-Net decoder. Fine-tuned ResNet-50 is kept as a second backbone for comparison, and the scratch CNN as the from-scratch baseline. All three checkpoints are stored in the shared Google Drive folder.

11 Failures and limitations

- **Scratch CNN overfits strongly** (96% train vs. 51% validation accuracy). Next: stronger augmentation such as Mixup/CutMix, a smaller or wider-but-shallower network, and longer training with a restart schedule.
- **The CNN without residual connections failed to train well** (44% training accuracy): this was an intended ablation, but it confirms that plain 18-layer CNNs are hard to optimise.
- **Tuning did not transfer to the full schedule**: the short-run optimum gave no improvement at 30 epochs. Next: tune with a longer proxy, include more hyper-parameters (augmentation strength, label smoothing, backbone LR multiplier), use several seeds, and try Optuna/Hyperband.
- **Single seed and a small validation set** (598 images): differences below ∼1.5 points are not statistically reliable.
- **Resolution 224 px** was chosen to fit 6–8 GB GPUs and the time budget; 448 px would likely gain 2–3 points.
- **Infrastructure**: the first Kaggle runs silently ran without GPU or internet because the account was not phone-verified; our CLI now refuses to train on a CPU unless explicitly allowed, and the code, data and weights are packaged so the jobs can run offline.
- **Label noise** (e.g. the nightjar pair) and genuinely ambiguous pairs (crow/raven) cap achievable accuracy.

12 Reproducibility and interface

All final code is pure Python, run through the CLI (`src/cli.py`); notebooks are not used for the final code. Exact commands:

```
python -m src.cli doctor          # checks GPU, CUDA PyTorch, W&B login, data
python -m src.cli prepare         # download, cache, split, deduplicate
python -m src.cli eda
python -m src.cli train --task classification --config configs/cls_convnext_tiny_ft.yaml
python -m src.cli tune  --config configs/tune_resnet50.yaml --method grid    # or random
python -m src.cli compare-tuning --dirs outputs/tuning/grid_* outputs/tuning/random_*
python -m src.cli evaluate --task classification --checkpoint <run>/best.pt --split test
python -m src.cli predict  --task classification --checkpoint <run>/best.pt --image bird.jpg
python -m src.cli gradcam  --checkpoints <scratch>/best.pt <resnet50>/best.pt --names scratch resnet50
python -m src.cli compare  --runs outputs/runs/*
```

Seeds are fixed, the validation split is stored in `data/processed/metadata.csv`, and every run folder and checkpoint contains the exact configuration it was trained with.

13 Team contributions

Member	Contribution
Abdelrahman Mohamed Zayed	Repository and project structure, data pipeline, EDA, training/tuning pipeline, Kaggle and W&B setup, experiments, report
Member 2	Data module: tests and documentation of caption cleaning, transforms and datasets
Member 3	Models: encoder/classifier tests, model summary script, architecture documentation
Member 4	Training: config and scheduler/optimizer tests, tuning tests, training documentation
Member 5	Evaluation: metric and Grad-CAM tests, batch-inference feature, evaluation documentation

References

[1] C. Wah, S. Branson, P. Welinder, P. Perona, S. Belongie. *The Caltech-UCSD Birds-200-2011 Dataset.* Caltech Tech. Report CNS-TR-2011-001, 2011.

[2] S. Reed, Z. Akata, H. Lee, B. Schiele. Learning Deep Representations of Fine-Grained Visual Descriptions. CVPR 2016.

[3] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. CVPR 2016.

[4] M. Tan, Q. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019.

[5] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, S. Xie. A ConvNet for the 2020s. CVPR 2022.

[6] R. R. Selvaraju et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017.

[7] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. JMLR 13, 2012.

[8] S. Kornblith, J. Shlens, Q. V. Le. Do Better ImageNet Models Transfer Better? CVPR 2019.

[9] Z. Yang et al. Learning to Navigate for Fine-grained Classification (NTS-Net). ECCV 2018. `https://arxiv.org/abs/1809.00287`

[10] Cut-Thumbnail: A Novel Data Augmentation for Convolutional Neural Network. ACM MM 2021. `https://arxiv.org/abs/2103.05342`

[11] T. Leemann et al. When are Post-hoc Conceptual Explanations Identifiable? UAI 2023. `https://arxiv.org/abs/2206.13872`

[12] Feature Mining: A Novel Training Strategy for Convolutional Neural Network. `https://arxiv.org/abs/2107.08421`

[13] J. He et al. TransFG: A Transformer Architecture for Fine-grained Recognition. AAAI 2022. `https://arxiv.org/abs/2103.07976`

[14] L. Biewald. Experiment Tracking with Weights and Biases, 2020. `https://wandb.ai`

[15] TorchVision maintainers. TorchVision: PyTorch's Computer Vision library. `https://github.com/pytorch/vision`